

Generalization

INFO 521 · Module 2 · Lecture A — Evaluation & Cross-Validation

Greg Chism

College of Information Science · University of Arizona

August 2026

Adapted with permission from lecture materials by Clayton Morrison, University of Arizona.

Learning outcomes

By the end of Lecture A you should be able to:

1. **Distinguish training error from generalization error.**
2. **Diagnose overfitting** from the gap between them.
3. **Run k -fold cross-validation.**
4. **Select a model or hyperparameter** by cross-validation.

Bridge from Module 1

In Lecture 1B we saw **basis functions** let us make the model *arbitrarily flexible* — swap $\mathbf{x} \rightarrow \phi(\mathbf{x})$, raise the degree.

- Flexibility **buys fit**: a high-degree curve can pass through every point.
- It also risks **memorizing** the sample instead of learning the pattern.

How much flexibility is *right*? That question is this whole module.

Overfitting, made visible

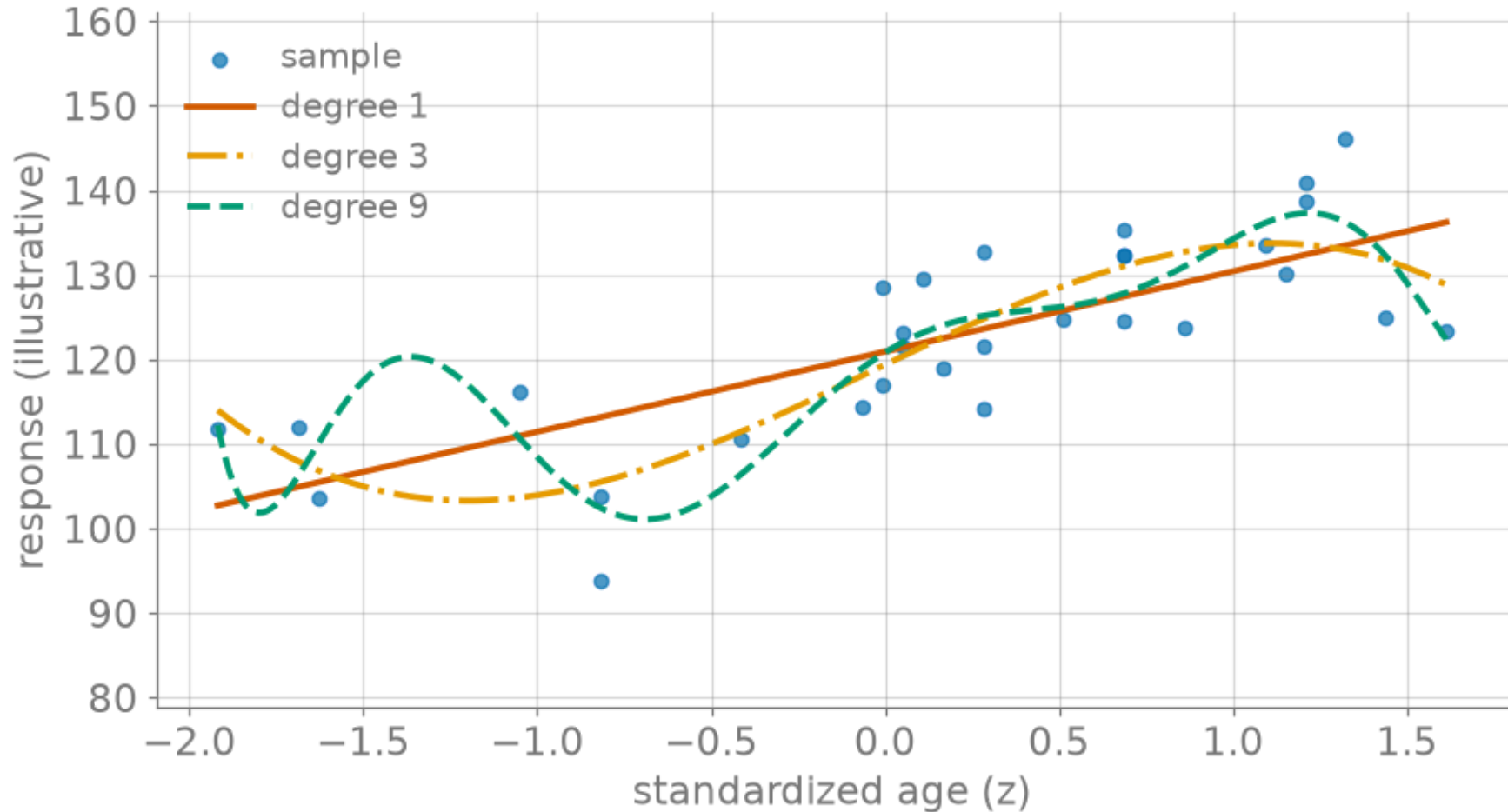


Figure 1: Illustrative signal, 30 points, three complexities: degree 1 (underfit), 3, 9 (overfit).

The trap: training error always drops

Make the model more flexible and the **training error only ever goes down** — a degree- k polynomial can interpolate $k+1$ points *exactly* ($\mathcal{L} = 0$).

- Low training error is **easy to buy** with complexity.
- It says nothing about new patients.

Training error is not generalization error.

Hold out a test set

Estimate generalization by scoring on data the model **never saw during fitting**:

- **Split** the patients into a **training** set (fit $\hat{\mathbf{w}}$) and a **test** set (score only).
- Test error is an honest estimate of error on *new* patients.

When we also tune a knob, a third **validation** (“dev”) set does the tuning, so the test set stays untouched until the very end.

Training vs. test vs. complexity

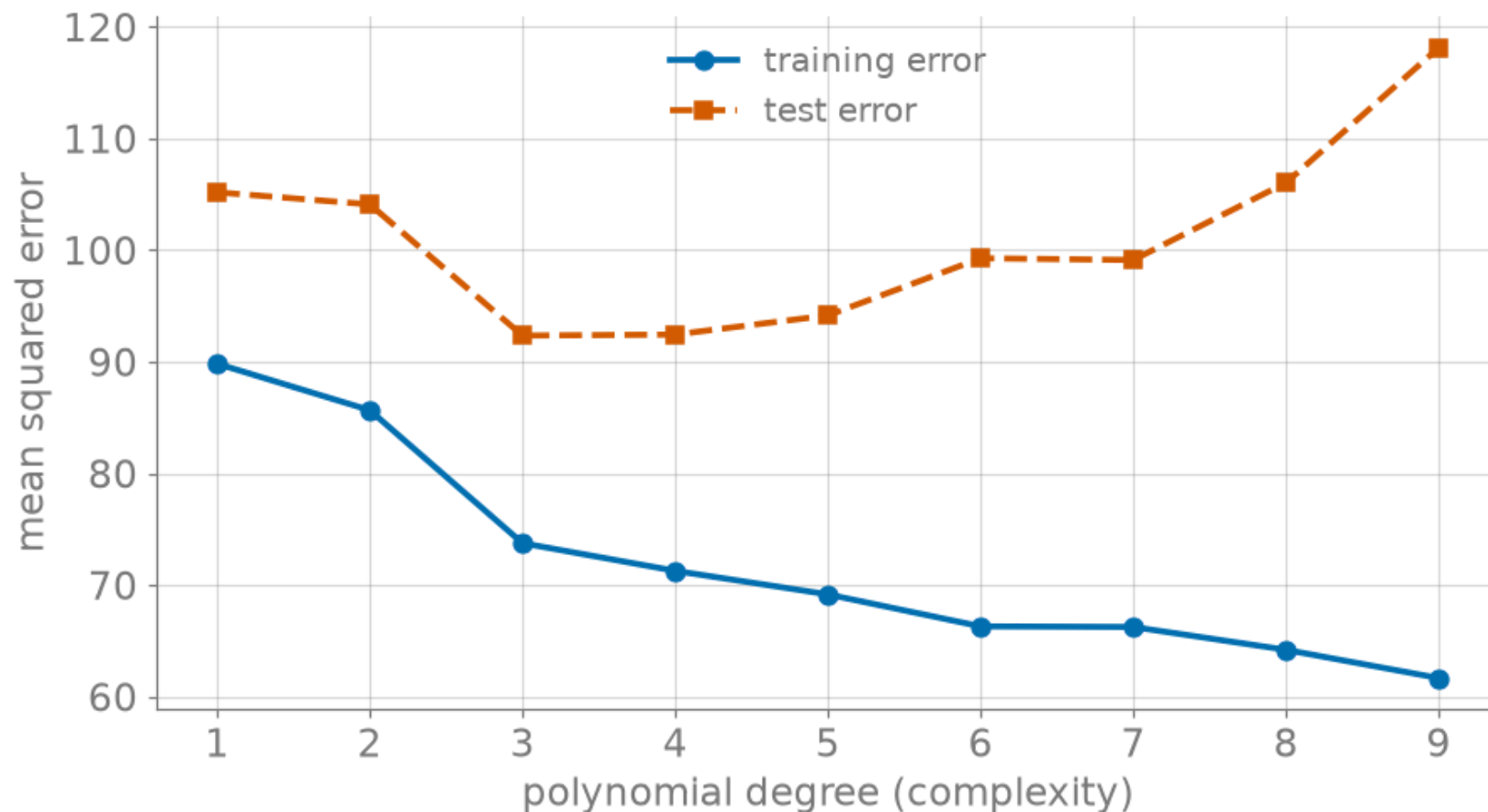


Figure 2: The central picture (illustrative signal, median over 30 training resamples): training error ↓; test error U-shaped.

The generalization gap

The distance between the two curves diagnoses the model:

- **Overfit** — low training error, high test error (the gap is wide, right side).
- **Underfit** — both high (left side); the model is too rigid to fit the signal.
- **Just right** — the bottom of the test-error U.

Why cross-validation?

A single train/test split has two problems, sharper at this sample size:

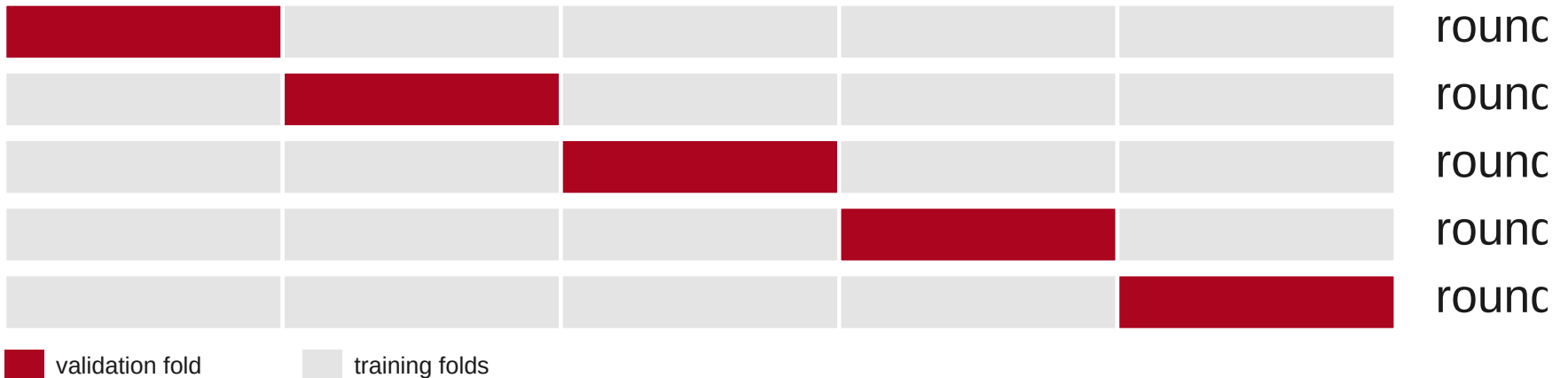
- it **wastes data** — every held-out patient is one you didn't train on;
- it is **high-variance** — a different split gives a different test error.

Cross-validation reuses every patient for both roles, in turn.

k -fold cross-validation

Partition into k folds; each fold takes a turn as the **validation** set while the other $k-1$ **train**; average the k scores.

5-fold: the validation block (accent) rotates across rounds



Cross-validation error curve

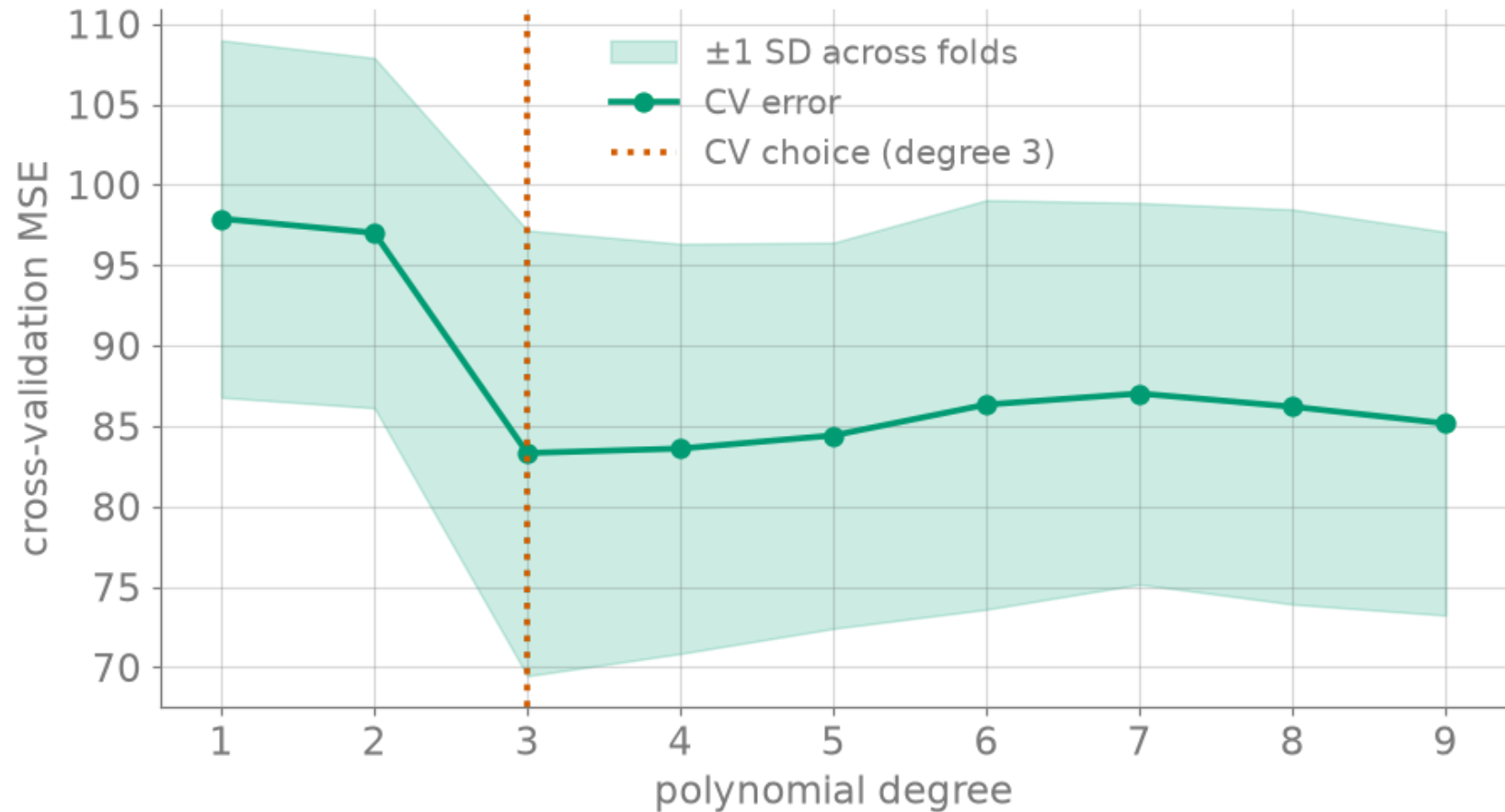


Figure 3: 5-fold CV error vs. degree (illustrative signal; band = ± 1 SD across folds); minimum marks the CV choice.

Model selection by CV

Pick the complexity that minimizes CV error. Then refit that model on *all* the training data and report on the held-out test set.

- CV is a *selection* tool — it chooses the knob.
- The final, untouched test set is what you *report*.

The pitfall: leakage

Cross-validation only tells the truth if the folds are **honestly separated**:

- **Preprocess inside each fold.** Standardize (center/scale) using the *training* fold's statistics, then apply to the validation fold — never on the pooled data.
- **Never tune on the test set.** Every peek leaks information and inflates the estimate.

Same discipline as the Module 1 feature-leakage rule — the answer must not sneak into the inputs.

Recap & bridge

- **Training error** falls with complexity; **test/CV error** is U-shaped.
- **k -fold CV** gives a stable estimate and picks the knob.
- **Leakage** breaks the estimate — separate folds honestly.

CV tells us **where** we are on the U. Next (Lecture B): **why** the U exists — the bias-variance decomposition — and how to **move along it** with regularization.